Solver Design Document

This app ships five independent Sudoku solvers:

1. **Backtracking (MRV)** — `js/sudoku-engine.js`, `solveGrid()`. Hand-written constraint-satisfaction search. Used by the app itself (puzzle generation, uniqueness checking, hints, the Solve button).
2. **CP — constraint propagation** — `js/cp-solver.js`, `solveGridCP()`. Sudoku as a CSP solved by domain-filtering propagation plus MRV-guided backtracking search when propagation stalls.
3. **DLX — Dancing Links (Algorithm X)** — `js/dlx-solver.js`, `solveGridDLX()`. Sudoku as an exact-cover problem, solved by Knuth's Algorithm X over a Dancing Links matrix.
4. **MILP — binary assignment** — `js/milp-solver.js`, `solveGridMILPBinary()`. Sudoku as a 0/1 integer program (the same exact-cover structure as DLX), solved by HiGHS.
5. **MILP — integer cells** — `js/milp-solver.js`, `solveGridMILPInteger()`. Sudoku as a general-integer program, also solved by HiGHS.

The CP, DLX, and MILP solvers are reference implementations for comparison, not wired into the main "Solve" button — see Comparison for why. All five run automatically on every new puzzle via the app's **Solver Comparison** panel (🧮 button in the top bar), which times and cross-checks them live.

All five solve the exact same problem instance and are expected to agree: given a partially-filled 9×9 grid (0 = empty) that has a unique solution, return the filled grid, or `null`/no result if none exists.

---

1. High-level comparison

	Backtracking (MRV)	CP (constraint propagation)	DLX (Dancing Links)	MILP — binary assignment	MILP — integer cells
Paradigm	Constraint-satisfaction search (depth-first + heuristic)	Constraint-satisfaction search (propagation + MRV backtracking)	Exact cover, solved by specialized backtracking (Algorithm X)	Integer linear programming (exact cover)	Integer linear programming (big-M all-different)

	Backtracking (MRV)	CP (constraint propagation)	DLX (Dancing Links)	MILP — binary assignment	MILP — integer cells
Decision variables	1 per empty cell, domain `{1..9}`	1 per cell, 9-bit candidate-set domain `{1..9}`	729 candidate placement rows over 324 constraint columns	729 binary `x[r][c][v]`	81 integer `y[r][c]` + up to 810 auxiliary binary `b`
Engine	Hand-written JS, this repo	Hand-written JS, this repo	Hand-written JS, this repo	HiGHS (C++, compiled to WASM)	HiGHS (C++, compiled to WASM)
Determinism	Deterministic (fixed cell/value ordering)	Deterministic (fixed propagation + MRV ordering)	Deterministic (fixed "S"-heuristic column ordering)	Depends on HiGHS's internal branching	Depends on HiGHS's internal branching
Typical time	**< 1 ms**	**< 0.2 ms**, essentially flat across difficulty	**< 0.3 ms**, essentially flat across difficulty	**~15–20 ms** (flat, low variance)	**~45 ms – 1+ s** (rises with difficulty, high variance)
Worst case observed	4.1 ms (hard)	0.3 ms (hard)	1.0 ms (easy)	24.5 ms (normal)	**5.1 s** (hard)
Async?	No	No	No	Yes (WASM)	Yes (WASM)
Used by the app for	Generation, uniqueness checks, hints, Solve button	Comparison panel only	Comparison panel only	Comparison panel only	Comparison panel only

---

2. Backtracking (MRV)

2.1 High-level idea

Sudoku is a constraint-satisfaction problem (CSP): fill in empty cells so that every row, column, and 3×3 box contains each of 1–9 exactly once. Standard depth-first backtracking tries a value in a cell, recurses, and undoes the choice if it leads to a dead end. The one refinement this solver uses over the naive version is the **Minimum Remaining Values (MRV)** heuristic: at each step, branch on whichever *empty cell currently has the fewest legal candidates*, rather than just the next empty cell in reading order. This fails fast — a cell with 0 candidates proves the current branch is dead immediately, and a cell with 1 candidate is a forced move — which prunes the search tree dramatically on sparse grids. See the "MRV-heuristic" explanation earlier in this project's chat history for the full walkthrough.

2.2 Formulation

As a CSP:

- **Variables:** `X[r][c]` for every cell `(r, c)`, `r, c ∈ {0, … ,8}`.
- **Domain:** `{1, … ,9}` for empty cells; a singleton `{given value}` for clue cells.
- **Constraints:** for every row, column, and 3×3 box `U` (a set of 9 cells), `AllDifferent(X[i] : i ∈ U)`.

There's no objective — it's a pure feasibility/CSP problem, solved by search rather than by relaxing to continuous variables.

2.3 Pseudocode

```
function solveGrid(grid):
    return backtrack(grid)

function backtrack(grid):
    cell, candidateCount = findMostConstrainedEmptyCell(grid)   # MRV
    if cell is None:
        return grid                      # no empty cells left → solved
    if candidateCount == 0:
        return FAIL                      # dead end, prune immediately
```

```
    for value in 1..9:
        if isSafe(grid, cell, value):    # row/col/box check
            grid[cell] = value
            result = backtrack(grid)
            if result ≠ FAIL:
                return result
            grid[cell] = 0               # undo and try the next value
    return FAIL

function findMostConstrainedEmptyCell(grid):
    best, bestCount = None, 10
    for each empty cell c in grid:
        n = count of values 1..9 that are safe at c
        if n == 0:
            return c, 0                  # can't do worse — stop scanning
        if n < bestCount:
            best, bestCount = c, n
            if n == 1:
                return best, 1           # can't do better — stop scanning
    return best, bestCount
```

This mirrors `sudoku-engine.js`'s `solveGrid()` / `findBestEmptyCell()` almost line for line — the real implementation just also tracks a solution counter (`countSolutions`) used for uniqueness checking during generation.

---

3. CP — constraint propagation

3.1 High-level idea

This is a genuinely different CSP strategy from backtracking (MRV), not just another implementation of it. Backtracking only ever asks "is this value locally safe *right now*?" at the moment it tries a value. CP instead keeps an explicit *domain* (candidate set) for every cell and propagates the consequences of each assignment immediately: the moment a cell is fixed, that digit is stripped from every peer's domain, which can force more cells, which forces more, and so on — much closer to how a human solves with pencil marks. Search (branching + backtracking) only kicks in once propagation alone can't decide anything further.

The propagation core follows the classic algorithm from Peter Norvig's "Solving Every Sudoku Puzzle", reimplemented here with 9-bit bitmask domains (one bit per candidate digit) instead of Python sets, for speed. Two inference rules, applied to a fixpoint via mutual recursion between an `assign` and an `eliminate` step:

- **Naked singles.** If eliminating a candidate leaves a cell with exactly one bit remaining, that digit can't appear anywhere else in the cell's row, column, or box — eliminate it from all 20 peers, which can itself trigger more naked singles.
- **Hidden singles.** Whenever a candidate is eliminated from a cell, check every unit (row/column/box) that cell belongs to: if some digit now has only one legal cell left *in that unit* — even if that cell's own domain still shows several candidates — it must go there.

Together these two rules are strictly stronger than plain arc consistency on a binary "not-equal" constraint (which is all the naked-single cascade alone would give): the hidden-single rule enforces a form of bounds consistency on the row/column/box `AllDifferent` constraints directly. In practice, most puzzles — including every difficulty this app generates — are solved by propagation alone or with only a handful of branch points.

When propagation stalls with more than one candidate left somewhere, the solver applies the same MRV heuristic as the backtracking solver: pick the cell with the fewest remaining candidates, try each one, re-propagate from that assumption, and recurse — undoing (via a domain-array copy) on contradiction. This "propagate, then branch, then propagate again" loop is what the CP literature calls **MAC** (Maintaining Arc Consistency).

3.2 Formulation

As a CSP — the variables and constraints are identical to the backtracking solver's; only the *algorithm* (propagation vs. plain search) differs:

- **Variables:** `X[r][c]` for every cell `(r, c)`, `r, c ∈ {0, … ,8}`, represented as a 9-bit mask over candidate digits rather than a single value.
- **Domain:** all 9 bits set for empty cells; a single bit for the given value in clue cells.
- **Constraints:** for every row, column, and 3×3 box `U`, `AllDifferent(X[i] : i ∈ U)`, enforced via the two propagation rules above rather than checked post hoc.

No objective, same as the other three — pure feasibility/CSP.

3.3 Pseudocode

```
function solveGridCP(grid):
    domains = 81 cells, each initialized to FULL (all 9 bits set)
    for each given clue (r, c, g):
        if not assign(domains, cell(r,c), bit(g)):
            return null                         # givens already contradictory
    result = search(domains)
    return decode(result) if result else null

function assign(domains, cell, bit):
    for each other candidate bit currently in domains[cell]:
        if not eliminate(domains, cell, bit):
            return false
    return true

function eliminate(domains, cell, bit):
    if bit not in domains[cell]: return true   # already gone
    domains[cell] −= bit
    if domains[cell] == empty: return false    # wiped out -- contradiction

    if domains[cell] has exactly one bit left:  # naked single
        onlyBit = domains[cell]
        for each peer of cell (row/col/box, 20 total):
            if not eliminate(domains, peer, onlyBit): return false

    for each unit (row/col/box) containing cell:  # hidden single
        places = cells in unit whose domain still contains bit
        if places.length == 0: return false       # nowhere left for this digit
        if places.length == 1:
            if not assign(domains, places[0], bit): return false
```

```
    return true

function search(domains):
    cell = the unsolved cell (domain size > 1) with fewest candidates  # MRV
    if no such cell: return domains             # every cell singleton → solved
    for each candidate bit in domains[cell]:
        trial = copy(domains)
        if assign(trial, cell, bit):
            result = search(trial)
            if result: return result
    return null                                  # every candidate dead-ended
```

The real implementation builds `domains` as a flat `Int`-array-backed bitmask table and precomputes units/peers once at module load — see `solveGridCP()` in `js/cp-solver.js` for the exact code, and `cp-test.html` for a standalone smoke test against the classic 30-clue example puzzle.

---

4. DLX — Dancing Links (Algorithm X)

4.1 High-level idea

Sudoku is an **exact cover** problem: choose a subset of candidate placements — "cell `(r,c)` holds digit `v`" — such that every constraint ("this cell holds *some* digit", "this row holds digit `v`", "this column holds digit `v`", "this box holds digit `v`") is satisfied by *exactly one* chosen candidate. That's precisely the structure `solveGridMILPBinary()` below encodes as a 0/1 integer program and hands to HiGHS's LP-relaxation + branch-and-cut machinery. This solver attacks the *same* exact-cover matrix directly, with no relaxation at all, using Donald Knuth's **Algorithm X** — a plain recursive backtracking search over the matrix — implemented with the **Dancing Links** (DLX) data structure that makes each step of that search fast.

The matrix: 729 candidate rows (one per `(cell, digit)` pair, restricted to a single row for any cell that's already given — that's the *entire* encoding of a clue, no special-casing needed anywhere else) × 324 constraint columns (81 cell + 81 row-digit + 81 column-digit + 81 box-digit). Each candidate row has exactly 4 ones, one per constraint it satisfies. "Solve the puzzle" = pick a set of rows that between them cover every column exactly once.

Algorithm X, plainly stated, is: pick any unsatisfied constraint (column), try each candidate row that could satisfy it, recursively solve what's left after removing that row and everything it conflicts with, backtrack if a branch dead-ends. The expensive part is bookkeeping — "remove this row and everything it conflicts with" touches many cells, and undoing it on backtrack normally means either a full re-scan or storing an explicit undo log. Dancing Links avoids both: every column header and every matrix cell lives in **two circular doubly-linked lists** at once — one running horizontally across the active columns, one running vertically within a column. "Covering" a column unlinks a handful of nodes by rewriting their neighbors' pointers (O(1) per removed node, not per matrix cell); crucially, nothing is ever *deleted* — a removed node's own pointers still point at its former neighbors. "Uncovering" replays those exact links in reverse, which is only correct because of that guarantee, and is what makes backtracking here as cheap as branching forward. (This is also the origin of the name: Knuth described undoing a removal as watching the deleted links "dance" back into place.)

The search itself uses Knuth's **"S" heuristic**: always branch on the column with the *fewest* remaining candidate rows — precisely the same minimum-remaining-values idea as the MRV backtracking solver's cell choice, just applied to constraints instead of cells. A column that reaches zero candidates is an immediate dead end, exactly like a cell with zero legal digits in the MRV solver.

4.2 Formulation

Identical exact-cover structure to `solveGridMILPBinary` (§5 below) — same variables, same constraints — solved by a specialized combinatorial search instead of an integer program:

- **Rows (candidates):** one per `(r, c, v)` triple with `v` a legal digit for cell `(r,c)` — all 9 digits for an empty cell, only the given digit for a clue.
- **Columns (constraints):** `cell(r,c)`, `row(r,v)`, `col(c,v)`, `box(b,v)` — 324 total, each requiring exactly one selected row to cover it.

No objective — pure feasibility, same as every other solver here.

4.3 Pseudocode

```
function solveGridDLX(grid):
    matrix = exact-cover matrix (see 4.2), linked via Dancing Links
    solution = search(matrix)
    return decode(solution) if solution else null

function search(matrix):
    if every column satisfied: return []          # solved
    col = the unsatisfied column with fewest candidate rows   # "S" heuristic
    if col has zero candidates: return FAIL        # dead end

    cover(col)
    for each row r with a node in col:
        for each other column j that r satisfies: cover(j)

        result = search(matrix)
        if result ≠ FAIL: return [r, ...result]

        for each other column j that r satisfies (reverse order): uncover(j)
    uncover(col)
    return FAIL

function cover(col):
    unlink col from the horizontal column list
    for each row r with a node in col:
        for each other column j that r satisfies:
            unlink r's node from column j's vertical list
```

```
function uncover(col):                              # exact reverse of cover()
    for each row r with a node in col (reverse order):
        for each other column j that r satisfies (reverse order):
            relink r's node back into column j's vertical list
    relink col back into the horizontal column list
```

The real implementation represents every column header and matrix cell as a plain JS object with `left` / `right` / `up` / `down` pointers (no external library — Dancing Links is small enough to hand-roll) — see `solveGridDLX()` in `js/dlx-solver.js` for the exact code, and `dlx-test.html` for a standalone smoke test against the classic 30-clue example puzzle.

---

5. MILP — binary assignment (exact cover)

5.1 High-level idea

This is the textbook ILP formulation for Sudoku — the same "exact cover" structure that backs algorithms like Knuth's Dancing Links. Instead of one variable per cell holding a digit, it uses one **binary** variable per *(cell, candidate digit)* pair: `x[r][c][v] = 1` means "cell `(r,c)` holds digit `v`". Every row/column/box/cell constraint becomes a simple linear equality: "exactly one of these binaries is 1." There's no `AllDifferent` to express — the exact-cover structure encodes it implicitly.

5.2 Mathematical formulation

Variables

$$x_{r,c,v} \in 0,1 \quad \forall r,c \in 0,\dots,8,\ v \in 1,\dots,9$$

(729 binary variables)

Constraints

$$\sum_{v=1}^{9} x_{r,c,v} = 1 \qquad \forall r,c \qquad \text{(one value per cell)}$$

$$\sum_{c=0}^{8} x_{r,c,v} = 1 \qquad \forall r,v \qquad \text{(each value once per row)}$$

$$\sum_{r=0}^{8} x_{r,c,v} = 1 \qquad \forall c,v \qquad \text{(each value once per column)}$$

$$\sum_{r \in \text{box}(br)} \sum_{c \in \text{box}(bc)} x_{r,c,v} = 1 \qquad \forall br,bc \in 0,3,6,\ v \qquad \text{(each value once per box)}$$

$$x_{r,c,g} = 1 \qquad \text{for every given clue } (r,c,g)$$

Objective: none — $\min 0$ (pure feasibility).

324 structural equality constraints + one fixed-bound per given clue (24–45 of them for this app's easy/normal/hard puzzles).

5.3 Pseudocode

```
function solveGridMILPBinary(grid):
    variables = {}
    constraints = {}

    for r, c in 0..8 x 0..8:
        constraints["cell_r_c"] = (sum of x[r][c][1..9] == 1)
    for v in 1..9:
        for r in 0..8: constraints["row_r_v"] = (sum of x[r][0..8][v] == 1)
        for c in 0..8: constraints["col_c_v"] = (sum of x[0..8][c][v] == 1)
        for each 3x3 box (br, bc):
            constraints["box_br_bc_v"] = (sum of x[r][c][v] for (r,c) in box == 1

    for r, c in 0..8 x 0..8:
        for v in 1..9:
            declare x[r][c][v] as binary
        if grid[r][c] is given as g:
            fix x[r][c][g] = 1              # via a Bounds entry, not an extra ro

    model = { minimize: 0, constraints, variables }
    result = HiGHS.solve(model)             # real branch-and-cut MILP solver
    if result.status ≠ Optimal:
        return null

    grid_out = empty 9x9 grid
    for r, c in 0..8 x 0..8:
        grid_out[r][c] = the v where x[r][c][v].value == 1
    return grid_out
```

The real implementation builds this as CPLEX `.lp` -format text (HiGHS's input format) rather than an in-memory object — see `solveGridMILPBinary()` in `js/milp-solver.js` for the exact string construction.

---

6. MILP — integer cells (big-M all-different)

6.1 High-level idea

A more "natural" formulation: one **integer** variable per cell holding the digit directly (`y[r][c] ∈ {1, … ,9}`), instead of 729 binary indicator variables. The catch: linear constraints can express equalities and inequalities, but not *disequality* (`y_i ≠ y_j`) directly — that's an inherently disjunctive condition (`y_i ≤ y_j − 1` OR `y_i ≥ y_j + 1`), and MILP has no native "or". The standard fix is a **big-M disjunction**: for every pair of cells sharing a unit, introduce one auxiliary binary variable that picks which side of the disjunction is enforced, with a constant `M` large enough to make the other side trivially slack.

6.2 Mathematical formulation

Variables

$$y_{r,c} \in 1, \dots, 9 \quad \forall r, c \qquad (\mathbf{81\ integer\ variables})$$

$$b_{ij} \in 0, 1 \quad \textbf{for each pair of cells } (i, j) \textbf{ sharing a unit}$$

Constraints — for every pair of cells $i = (r_1, c_1)$, $j = (r_2, c_2)$ in the same row, column, or box (deduplicated — a pair sharing both a row *and* a box only gets one disjunction, not two):

$$y_i - y_j + M \cdot b_{ij} \geq 1 \qquad (\mathbf{A})$$

$$y_j - y_i + M \cdot (1 - b_{ij}) \geq 1 \qquad (\mathbf{B})$$

$b_{ij} = 0$ makes (A) binding (forces $y_i > y_j$) while (B) goes slack; $b_{ij} = 1$ flips it. Either way $y_i \neq y_j$. M must satisfy $M \geq \mathbf{max}(y) - \mathbf{min}(y) = 8$; $M = 9$ is the smallest integer that keeps the "slack" side's worst case ($1 - M = -8$) from being violated by the true extreme ($y_j - y_i = 1 - 9 = -8$) — this exact off-by-one (an earlier draft used $M = 8$) was caught by independently re-checking the constraint math against a known correct solution before this module was integrated.

Plus bounds: $1 \leq y_{r,c} \leq 9$ for empty cells, $y_{r,c} = g$ for every given clue g. Pairs where **both** cells are already given are skipped entirely — two fixed, distinct values need no disjunction, which keeps the model's size proportional to how many cells are still unsolved rather than always paying for the full ~810 pairs.

Objective: none — $\mathbf{min}\, 0$, same as the binary formulation.

6.3 Pseudocode

```
function solveGridMILPInteger(grid):
    M = 9
    variables = {}
    constraints = {}
    auxiliary_binaries = []

    for r, c in 0..8 x 0..8:
        declare y[r][c] as a general integer variable
        if grid[r][c] is given as g:
            constraints["bound_r_c"] = (y[r][c] == g)
        else:
            constraints["bound_r_c"] = (1 ≤ y[r][c] ≤ 9)

    for each unit U in {rows, columns, boxes}:
        for each unordered pair (i, j) of cells in U, deduplicated:
            if grid[i] and grid[j] are both given:
                continue                    # trivially different already
            b = new binary variable
            auxiliary_binaries.append(b)
            constraints["diff_a"] = (y[i] − y[j] + M*b ≥ 1)
            constraints["diff_b"] = (y[j] − y[i] − M*b ≥ 1 − M)

    model = { minimize: 0, constraints, variables, integers: all y,
              binaries: auxiliary_binaries }
    result = HiGHS.solve(model)
    if result.status ≠ Optimal:
        return null

    grid_out = empty 9x9 grid
    for r, c in 0..8 x 0..8:
        grid_out[r][c] = round(y[r][c].value)
    return grid_out
```

---

7. Comparison: quality & performance

7.1 Quality / correctness

All five solvers are **exact and complete** — no approximation, no heuristic risk of returning a wrong or partial answer. If a solution exists, each one is guaranteed to find *a* solution; if the grid is infeasible, each correctly reports that (`null` /non-optimal status) rather than returning garbage.

Verified directly, not just claimed: all five were run against puzzles generated by the app's own generator (which independently verifies uniqueness via `countSolutions` before returning), and their outputs were compared against that known-correct solution with `gridsEqual` . Across every generated easy/normal/hard puzzle tested, **all five agreed with the known solution every time** — this is also what the in-app Solver Comparison panel checks and displays live (`matches solution ✓/✗`) for every new puzzle, not just at build time. The CP and DLX solvers were each additionally cross-checked against the backtracking solver directly (30/30 identical results on hard puzzles for both) and against a deliberately corrupted grid (two givens conflicting in the same row), which both correctly report as infeasible (`null`). DLX was further checked against a fully empty grid (no givens at all) and still returned a genuinely valid Sudoku.

One subtlety worth flagging: **none of the five enforce solution uniqueness** — they each just return *a* feasible solution. For a puzzle this app actually generates, the solution is unique by construction, so "a solution" and "the solution" coincide. Fed a puzzle with multiple valid solutions, the five solvers are not guaranteed to return the *same* one (they search/branch differently) — this was confirmed directly during testing with a malformed (mistyped, non-unique) 17-clue puzzle: the backtracking solver and both MILP solvers each returned a different valid completion, none of them "wrong", just different valid solutions to an under-constrained puzzle. The CP and DLX solvers were separately confirmed to do the same on an even more under-constrained grid (a single given clue) — each always returns a valid, fully consistent completion, just not necessarily matching any other solver's particular choice.

7.2 Performance

Measured live in-browser (Chromium via Playwright), against puzzles from the app's own generator — 100 runs per difficulty for backtracking, CP, and DLX (all pure JS, negligible per-call overhead, JIT-warmed first), 5 runs per difficulty for the MILP solvers (each MILP run pays a fresh HiGHS branch-and-cut solve; the one-time ~3.4MB WASM load cost is excluded, since in the app it's paid once per page load and cached — see `getHighs()` in `js/milp-solver.js`):

Difficulty (clues)	Backtracking (MRV)	CP	DLX	MILP — binary	MILP — integer
Easy (38–45)	avg **0.045 ms**, max 0.3 ms	avg **0.09 ms**, max 0.2 ms	avg **0.22 ms**, max 1.0 ms	avg **16.8 ms**, range 14.0–20.1 ms	avg **44.5 ms**, range 32.6–50.3 ms
Normal (30–37)	avg **0.11 ms**, max 0.5 ms	avg **0.09 ms**, max 0.2 ms	avg **0.21 ms**, max 0.6 ms	avg **20.4 ms**, range 17.1–24.5 ms	avg **115.3 ms**, range 85.8–172.1 ms
Hard (24–29)	avg **0.55 ms**, max 3.5 ms	avg **0.10 ms**, max 0.3 ms	avg **0.18 ms**, max 0.5 ms	avg **18.7 ms**, range 16.5–20.8 ms	avg **1120.6 ms**, range 97.7 ms – **5.12 s**

(Earlier easy/normal/hard backtracking figures in this table, from a smaller 20-run sample, were 0.13/0.32/1.10 ms average — consistent with the larger sample above once JIT warm-up noise is excluded.)

Observations:

- **CP, DLX, and backtracking are all effectively free**, and CP/DLX's averages are essentially flat across difficulty (~0.1–0.2 ms) while backtracking's rises with difficulty (0.045 ms → 0.55 ms average, up to 3.5 ms worst case, and occasional double-digit-ms outliers observed separately). This is exactly what a propagation-/structure-first design predicts: CP resolves most of the board through naked/hidden-single deduction before branching at all, and DLX's Dancing Links bookkeeping keeps every branch/undo O(1) regardless of how sparse the grid is, so neither one notices that harder puzzles have fewer givens the way plain backtracking does. On the hardest puzzles, CP and DLX are now the fastest solvers in the app, edging out even the hand-tuned MRV backtracking search.
- **Backtracking, CP, and DLX all win outright** for anything performance- sensitive: 2–4 orders of magnitude faster than either MILP solver, no async/WASM overhead. Backtracking remains what the app actually uses in the critical path (generation, hints, Solve), since it's already there and the difference from CP/DLX is negligible in absolute terms.
- **DLX is essentially the "compiled" version of the MILP binary formulation** — literally the same exact-cover matrix (§4.2 vs. §5.2), but walked with pointer surgery on a linked-list structure purpose-built for exact cover instead of relaxed into an LP and handed to a general-purpose branch-and-cut solver. The ~100x speed gap between the two here (DLX at a fraction of a millisecond vs. MILP binary at ~15–20ms) is a direct, concrete illustration of what a problem-specific algorithm buys you over a general-purpose one on the exact same formulation — HiGHS is doing real work (presolve, cuts, simplex pivots) that Dancing Links simply has no need for.

- **MILP binary assignment is flat and predictable** — ~15–20ms regardless of difficulty, with low variance. This tracks the formulation's structure: it's the same clean exact-cover polytope that specialized algorithms exploit, and HiGHS's presolve/cuts handle it efficiently and consistently.
- **MILP integer cells is the outlier** — usually competitive on easy puzzles, but degrades sharply with difficulty and has high variance, including one 5.12-second run on a hard puzzle. This matches the theory: fewer givens means more free-cell pairs needing a big-M disjunction (up to 810 auxiliary binaries), and big-M constraints are a notoriously loose LP relaxation — branch-and-bound has to work much harder to close the gap. An earlier, smaller-sample measurement (2 puzzles) had suggested this formulation was competitive with or even faster than the binary one; the broader 5-run-per-difficulty sample above shows that was not representative — the integer formulation is consistently slower on average and dramatically worse in the worst case.
- Both MILP solvers were, separately, benchmarked against the *previous* pure-JS solver backend (`javascript-lp-solver`) before this app switched to HiGHS: that backend took **minutes-plus per solve, often not finishing at all**, on the exact same formulations — a symptom of Sudoku's exact-cover constraint structure being classically degenerate for plain dense-tableau simplex without presolve/cuts. HiGHS's presolve, cutting planes, and degeneracy-aware pivoting are the entire reason both MILP solvers are usable at all now.

7.3 Recommendation

Use **backtracking (MRV)**, **CP**, or **DLX** for anything performance- sensitive or in the critical path — all three are effectively free, and the app already depends on backtracking for generation/uniqueness-checking regardless, so that's what stays wired to the Solve button. CP and DLX each earn their place as more than a curiosity, though, for different reasons:

- **CP** is the closest thing in this repo to how a human actually solves — propagate what you can deduce for certain, only guess when truly stuck. If a future feature wants solving *steps* to explain to a player ("this cell is forced because it's the only place left for a 7 in this box"), CP's naked/hidden-single propagation already produces exactly that trace; plain backtracking doesn't reason in those terms at all.
- **DLX** is the most *consistent* performer of the three on the numbers above (flat ~0.1–0.2 ms regardless of difficulty) and, more importantly, is a direct, concrete demonstration of what a problem-specific data structure buys you: it solves the *exact same* exact-cover matrix as `solveGridMILPBinary` roughly 100x faster, with no LP relaxation, no external solver, and no WASM. If a future Sudoku variant's constraints still reduce to exact cover (most do — even killer-Sudoku cages can be encoded as extra columns), DLX is worth reaching for before MILP.

The two MILP solvers earn their place as a correctness cross-check and a worked demonstration of two different ILP formulations of the same problem (exact-cover vs. big-M all-different) and how much a real solver backend (HiGHS) versus a naive one changes their practicality — not as a faster alternative to CP/DLX for Sudoku itself. Where MILP (or the LP-relaxation machinery behind it) genuinely earns its keep is a variant whose constraints *don't* reduce cleanly to exact cover or plain backtracking — e.g. one with real linear/inequality constraints (weighted regions, numeric sum constraints à la Killer Sudoku) rather than pure "all-different." Even then, prefer the binary/exact-cover style formulation over big-M when the problem does still fit that mold — the numbers above are a direct demonstration of why.